

502

03AAX Algoritmi e Strutture Dati

Appello del 11/05/2023 - Prova di Teoria (12 punti)

1. (2.5 punti)

Si risolva la seguente equazione alle ricorrenze mediante il metodo dello sviluppo (unfolding):

$$\begin{aligned} T(n) &= 3T(n/4) + 3n & n > 1 \\ T(1) &= 1 & n = 1 \end{aligned}$$

2. (2.0 punti)

Sia data la sequenza di interi, supposta memorizzata in un vettore:

7 2 6 31 3 67 11 18 8 41 0 5 9 10

la si trasformi in un heap, ipotizzando di usare un vettore come struttura dati. Si riportino graficamente i passi significativi della costruzione dell'heap ed il risultato finale. Si ipotizzi che, alla fine, nella radice dell'heap sia memorizzato il valore minimo si eseguano su tale heap i primi 2 passi dell'algoritmo di heapsort.

NB: la sequenza è già memorizzata nel vettore e rappresenta una configurazione intermedia per cui la proprietà di heap non è ancora soddisfatta.

3. (2.5 punti)

Data la catena di matrici (A_1, A_2, A_3, A_4) di dimensioni (3×4) , (4×2) , (2×5) e (5×3) rispettivamente, si determini mediante un algoritmo di programmazione dinamica la parentesizzazione ottima del prodotto di matrici che minimizza il numero di moltiplicazioni.

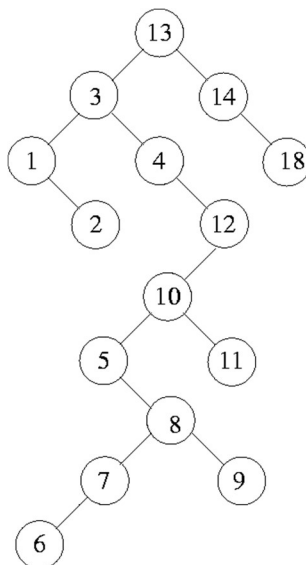
4. (1 punto)

Si converta la seguente espressione da forma prefissa a forma postfissa:

/ * - A B / C D / E * - F G + H I

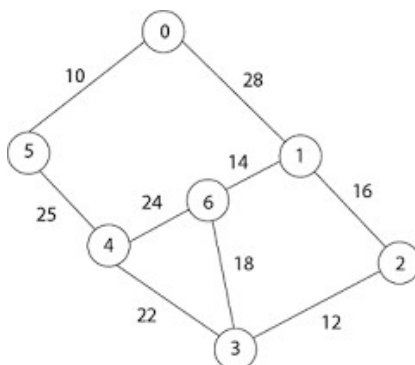
5. (2.0 punti)

Si partizioni il seguente BST attorno alla sesta chiave più piccola. In ogni nodo compare la chiave intera, altre eventuali informazioni non sono riportate.



6. (2.0 punti)

Dato il seguente grafo non orientato, connesso e pesato, se ne determini un minimum spanning tree applicando l'algoritmo di Prim a partire dal vertice 0, disegnando l'albero e ritornando come risultato il valore del peso minimo. Si esplicitino i passi intermedi.



1. (2.5 punti)

Si risolva la seguente equazione alle ricorrenze mediante il metodo dello sviluppo (unfolding):

$$\begin{aligned} T(n) &= 3T(n/4) + 3n & n > 1 \\ T(1) &= 1 & n = 1 \end{aligned}$$

Equazione alle ricorrenze

1. Esegui 3 iterazioni dell'equazione alle ricorrenze.

a. Divide and conquer:

$$\begin{cases} T(n) = T\left(\frac{n}{x}\right) + c(n) & n > 1, x > 2 \\ T(1) = 1 & n = 1 \end{cases}$$

b. Decrease and conquer:

$$\begin{cases} T(n) = T(n-x) + c(n) & n > 1, x > 2 \\ T(1) = 1 & n = 1 \end{cases}$$

2. Sostituisci la terza equazione ottenuta nella seconda, successivamente sostituire la seconda ottenuta nella prima.

3. Trovare una legge che può essere definita attraverso una sommatoria, e definire l'indice alla quale terminerà.

a. Divide and conquer:

$$\frac{n}{x^i} = 1 \Rightarrow i = \log_x n$$

b. Decrease and conquer:

$$n - ix = 1 \Rightarrow i = \frac{n-1}{x}$$

4. Risolvere la sommatoria ottenuta.

5. Il valore più alto ottenuto (non negativo) rispetto ad n sarà quello utilizzato per la definizione della complessità.

$$\frac{n}{x^i} = 1 \Rightarrow \frac{n}{4^i} = 1 \quad i = \log_4 n$$

$$T(n) = 3T\left(\frac{n}{4}\right) + 3n \quad n > 1 \quad T(1) = 1$$

• Divide and Conquer

$$1) \cdot T\left(\frac{n}{4}\right) = 3T\left(\frac{n}{16}\right) + 3\left(\frac{n}{4}\right)$$

$$\cdot T(n) = 3T\left(\frac{n}{4}\right) + 3n$$

$$\cdot T(n) = 3 \left[3T\left(\frac{n}{16}\right) + 3\left(\frac{n}{4}\right) \right] + 3n$$

$$\cdot T(n) = 3^2 T\left(\frac{n}{16}\right) + 3n\left(\frac{3}{4}\right) + 3n$$

$$2) \quad T\left(\frac{n}{16}\right) = 3T\left(\frac{n}{64}\right) + 3\frac{n}{16} \quad \left| \quad T(n) = 3^2 \left[3T\left(\frac{n}{64}\right) + 3\left(\frac{n}{16}\right) \right] + 3n\left(\frac{3}{4}\right) + 3n$$

$$T(n) = 3^3 T\left(\frac{n}{64}\right) + 3^3 \left(\frac{n}{16}\right) + 3n\left(\frac{3}{4}\right)^2 + 3n \Rightarrow T(n) = 3^3 T\left(\frac{n}{64}\right) + 3n\left(\frac{3}{4}\right)^2 + 3n\left(\frac{3}{4}\right)^1 + 3n\left(\frac{3}{4}\right)^0$$

Formula

$$T(n) = 3^K T\left(\frac{n}{4^K}\right) + 3n \sum_{i=0}^{K-1} \left(\frac{3}{4}\right)^i$$

• (condizione di arresto) $\frac{n}{4^K} = 1 \quad K = \log_4 n$
($n = 1$)

$$3^{\log_4 n} + 3n \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{4}\right)^i$$

• Sym.

$$\sum_{i=0}^{N-1} \left(\frac{3}{4}\right)^i = \frac{1 - \left(\frac{3}{4}\right)^N}{1 - \frac{3}{4}} \rightarrow 4 \left[1 - \left(\frac{3}{4}\right)^N \right]$$

$$4 \left[1 - \frac{3^{\log_4 n}}{4^{\log_4 n}} \right] \rightarrow 4 \left(1 - \frac{\log_4 3}{n} \right)$$

• Now we have

$$T(n) = \underbrace{n^{\log_4 3}}_{\text{blue}} + \underbrace{3n \left[4 \left(1 - \frac{\log_4 3}{n} \right) \right]}_{\text{red}}$$

$$= n^{\log_4 3} + 12n - 12n^{\log_4 3}$$

$$= 12n - 11n^{\log_4 3} \sim 0.79$$

• $n > n^{\log_4 3} \rightarrow T(n) = \Theta(n)$ (Then we have 0)

• CLAR (A) THEOREM MASTER

$$T(n) = 35 \left(\frac{n}{4} \right) + 3n$$

$$a = 3, b = 4$$

$$f(n) = 3n$$

$$n > n^{0.79}$$

$$n^{\log_4 3} = n^{\log_4 3} \sim n^{0.79}$$

2. (2.0 punti)

Sia data la sequenza di interi, supposta memorizzata in un vettore:

7 2 6 31 3 67 11 18 8 41 0 5 9 10

la si trasformi in un heap, ipotizzando di usare un vettore come struttura dati. Si riportino graficamente i passi significativi della costruzione dell'heap ed il risultato finale. Si ipotizzi che, alla fine, nella radice dell'heap sia memorizzato il valore minimo si eseguano su tale heap i primi 2 passi dell'algoritmo di heapsort.

NB: la sequenza è già memorizzata nel vettore e rappresenta una configurazione intermedia per cui la proprietà di heap non è ancora soddisfatta.

Trasformazione in heap

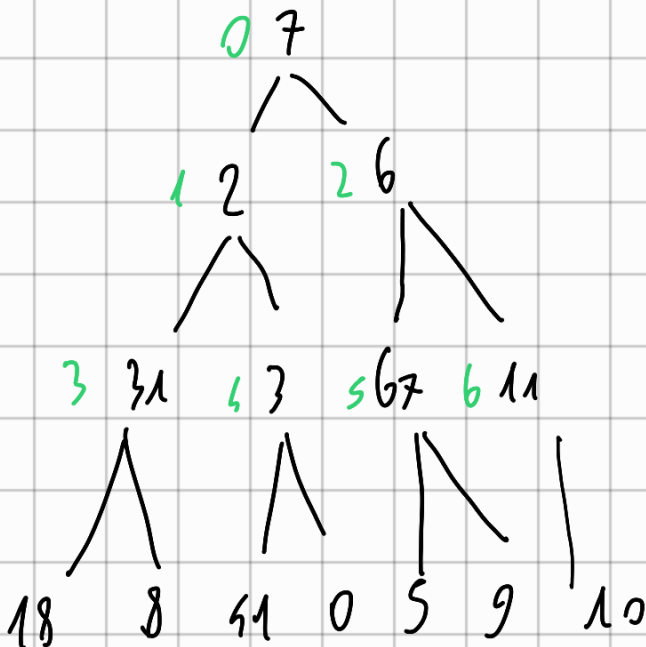
1. Le foglie sono già heap. Si parte dai padri delle foglie.
2. Si confronta la radice con il figlio sinistro.
 - a. Se l'elemento è minore del figlio sinistro, allora si propone il figlio sinistro come nuova radice del sotto-albero.
 - b. Altrimenti si propone l'elemento come radice del sotto-albero.
3. Si confronta l'elemento candidato con il figlio destro.

- a. Se l'elemento candidato è minore del figlio destro, allora si propone il figlio destro come nuova radice del sotto-albero.
 - b. Altrimenti si propone l'elemento come radice del sotto-albero.
4. L'elemento candidato diventa la radice del sotto-albero, sostituendo la radice attuale.
 5. Si ripete con tutti i nodi dal basso verso l'alto.

Heapsort

1. Si scambia l'elemento alla radice dell'albero con l'ultimo elemento presente nell'heap, e lo si salva in un vettore alla posizione i , dove i indica il numero del passo da 0 a n .
2. Si applica la trasformazione in heap, mantenendo l'elemento alla fine dell'heap appena scambiato nella stessa posizione.
3. Applica questo procedimento per ogni elemento fino ad ottenere l'ordinamento.

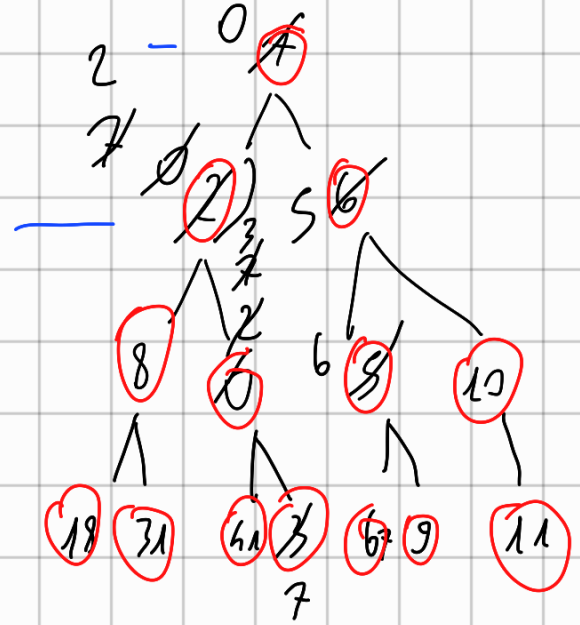
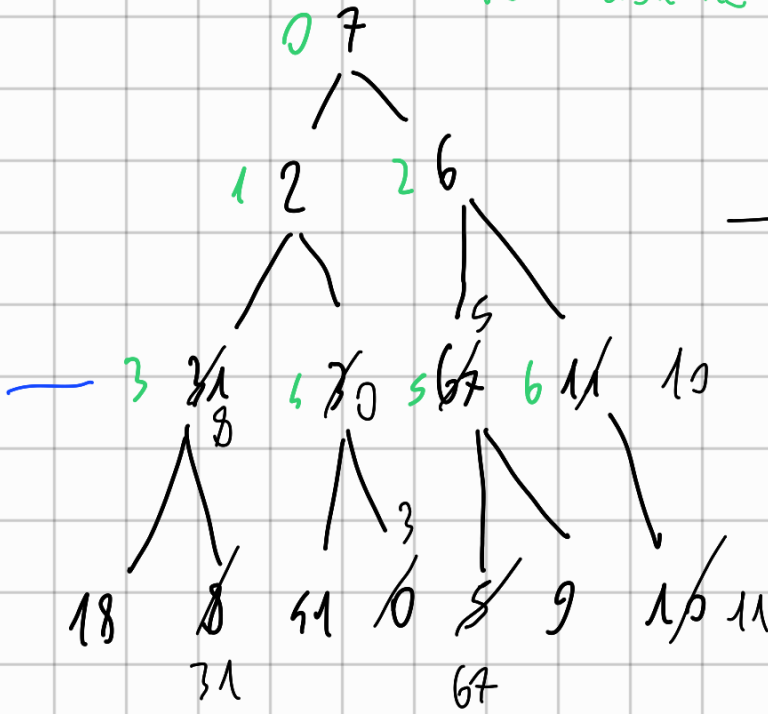
7 2 6 31 3 67 11 18 8 41 0 5 9 10 | h: 14
[0, 13]



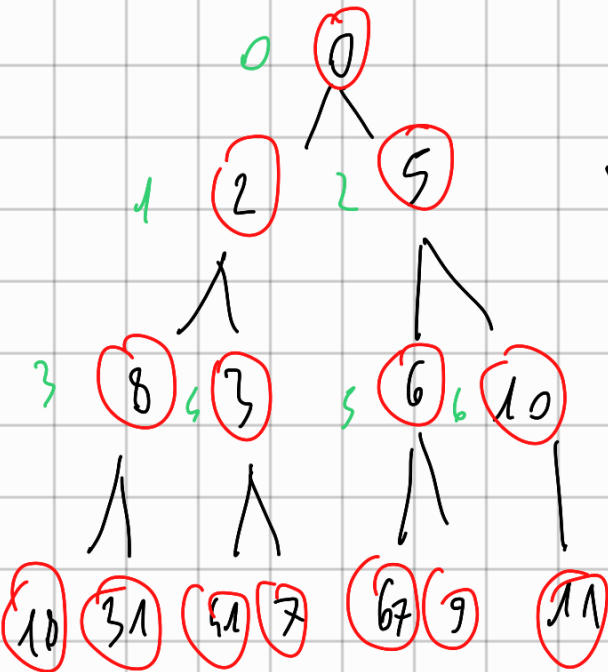
Ultimo nodo, non foglia: $\left\lceil \frac{n}{2} \right\rceil - 1 = 6$
[0, 6]

← BUILD THE HEAP

Devo SISTEMARE: HEAP



↙

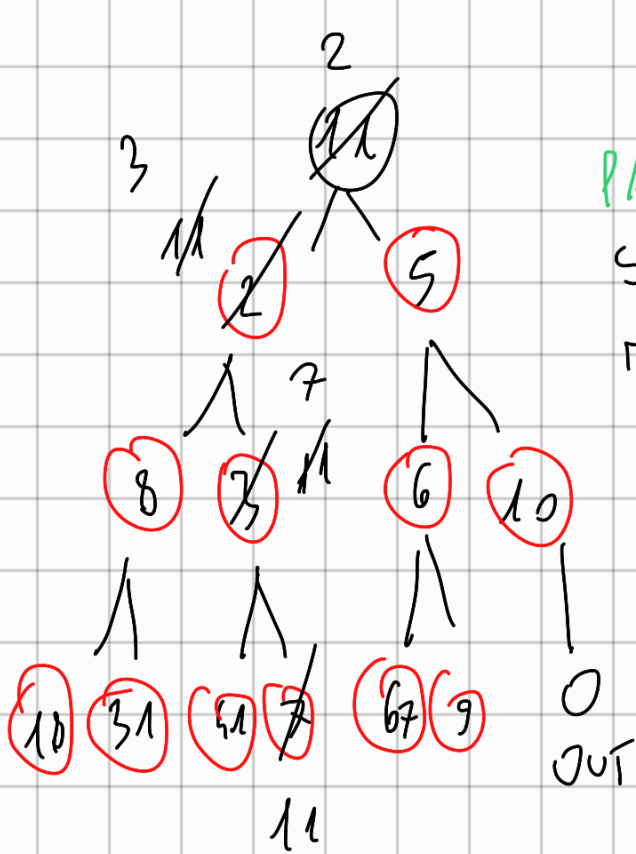


LISTE [0, 2, 5, 8, 3, 6, 10, 18, 31, 41, 7, 67, 9, 11]

HEAP INIZIALE

Q

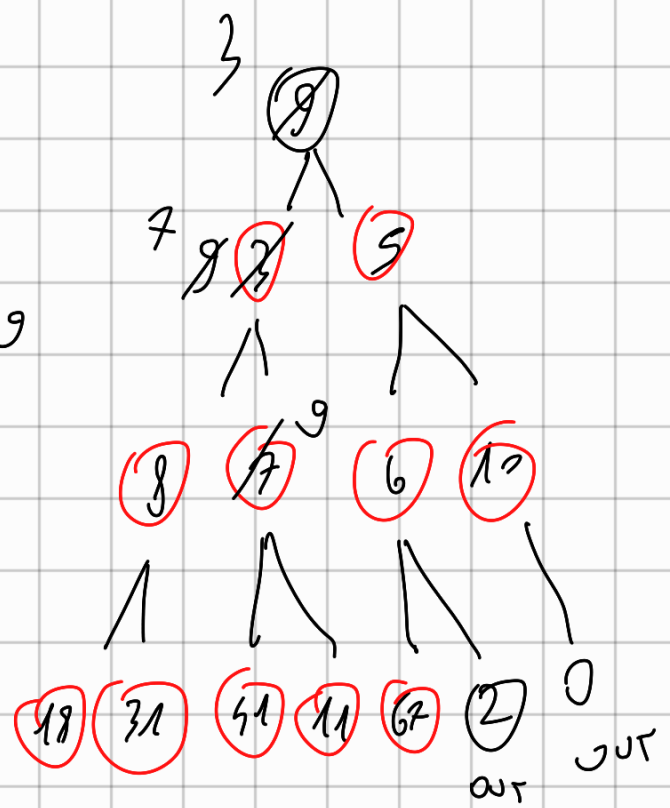
ASSO 1 HEAP SORT: LISTA MIN (0) LA SOTTO CON ULTIMO (11)



PASSO 2

SWAP

MIN(2) $\leftrightarrow$ 9



HEAD

FINAL

HEAD PASSO 1

[2, 3, 5, 8, 7, 6, 10, 18, 31, 41, 7, 67, 9, 10] [3, 4, 5, 8, 9, 6, 10, 18, 31, 41, 11, 67, 2, 10]

3. (2.5 punti)

Data la catena di matrici (A_1, A_2, A_3, A_4) di dimensioni (3×4) , (4×2) , (2×5) e (5×3) rispettivamente, si determini mediante un algoritmo di programmazione dinamica la parentesiizzazione ottima del prodotto di matrici che minimizza il numero di moltiplicazioni.

- Definisci le matrici $A_1, A_2, \dots, A_n$, di dimensioni $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$, con $p_0, p_1, \dots, p_n$ dimensioni delle matrici.
- Disegna la matrice superiore indicizzata da 1 a n per la valutazione della parentesiizzazione ottima, con le prime n celle con valore 0.
- Disegna la matrice superiore indicizzata da 2 a n per la stampa della soluzione, con le prime $n-1$ celle con valori da 1 a $n-1$.
- Definiamo i valori della parentesiizzazione per intervalli di dimensione 2, con $k = 1 + x + y$, con $x = \{0, 1, \dots, n-1\}$ e $y = \{0, 1, \dots, n-1\}$ e prendere i valori minori tra le scelte, inserendole all'indice $(2 + y, 1 + y)$ nella prima matrice.

$$m_{k(k+1)} = m_{kk} + m_{(k+1)(k+1)} + p_{k-1}p_kp_{k+1}$$

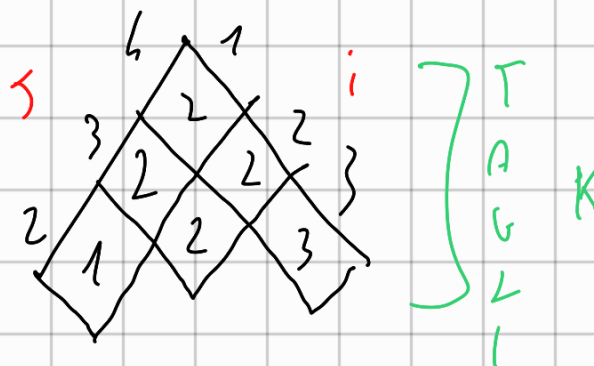
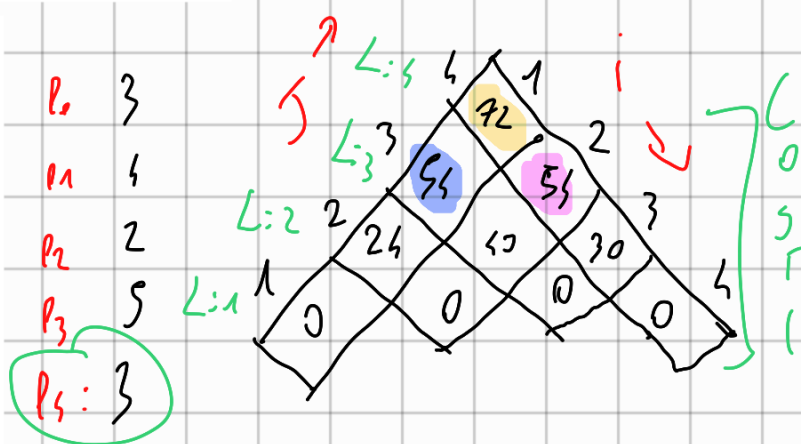
Nella seconda matrice, inserire l'indice k alla posizione $(2 + y, 1 + y)$.

- Definiamo i valori della parentesiizzazione per intervalli di dimensione 3, con $k = 1 + x + y$, con $x = \{0, 1, \dots, n-2\}$ e $y = \{0, 1, \dots, n-2\}$ e prendere i valori minori tra le scelte, inserendole all'indice $(3 + y, 1 + y)$.

$$m_{k(k+2)} = \begin{cases} m_{kk} + m_{(k+1)(k+2)} + p_{k-1}p_kp_{k+2} \\ m_{k(k+1)} + m_{(k+2)(k+2)} + p_{k-1}p_{k+1}p_{k+2} \end{cases}$$

Nella seconda matrice, inserire l'indice k alla posizione $(3 + y, 1 + y)$.

- Ripetere fino a riempire le due matrici.
- Per stampare la soluzione, a partire dall'elemento $(n, 1)$, definendo il primo indice come i e il secondo indice come j , calcolare il valore $m = \lfloor (i + j) / 2 \rfloor$.
 - Se $i \neq j$, ricorrere sugli indici (m, i) e $(j, m + 1)$.
 - Se $i = j$, stampare la matrice con indice i .
 - Ripetere fino a terminazione dell'albero di ricorsione.



$L=1$ (diagonali principali)

$$m_{11} = 0$$

$$m_{22} = 0$$

$$m_{33} = 0$$

$$m_{44} = 0$$

$$m_{k(k+1)} = m_{kk} + m_{(k+1)(k+1)} + p_{k-1}p_kp_{k+1}$$

$L=2$

$$m_{12} = p_0p_1p_2 = 3 \cdot 4 \cdot 2 = 24 \quad K=1$$

$$m_{23} = p_1p_2p_3 = 4 \cdot 2 \cdot 5 = 40 \quad K=2$$

$$m_{34} = p_2p_3p_4 = 2 \cdot 5 \cdot 3 = 30 \quad K=3$$

$L=4$

$$m_{14} = m_{11} + m_{24} + p_0p_1p_4 = 90 \quad K=1$$

$$m_{14} = m_{12} + m_{34} + p_0p_2p_4 = 72 \quad K=2$$

$$m_{14} = m_{13} + m_{44} + p_0p_3p_4 = 99 \quad K=3$$

$L=3$

5311

$$m_{13} = m_{11} + m_{23} + p_0p_1p_3 = 100 \quad K=1$$

$$m_{13} = m_{12} + m_{33} + p_0p_2p_3 = 54 \quad K=2$$

$$m_{24} = m_{22} + m_{34} + p_1p_2p_4 = 54 \quad K=2$$

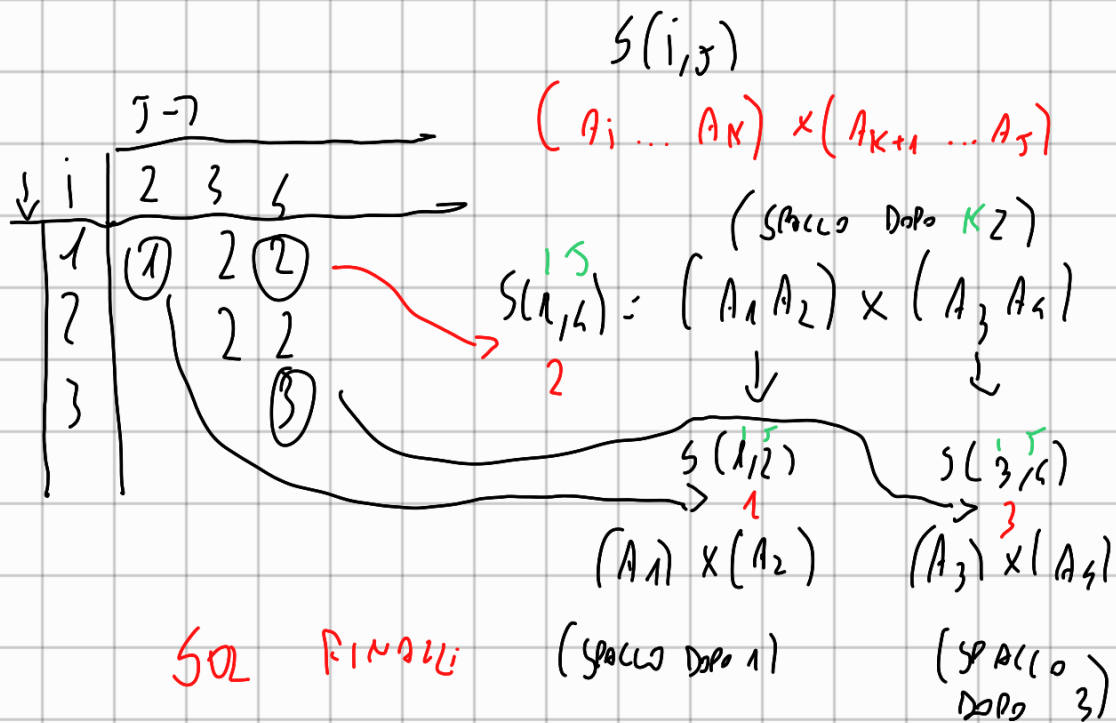
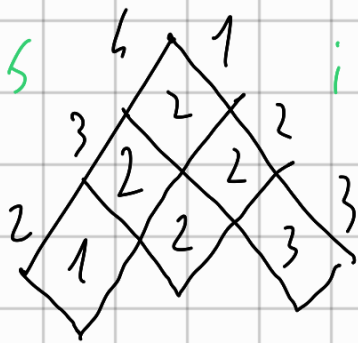
$$m_{24} = m_{23} + m_{44} + p_1p_3p_4 = 100 \quad K=3$$

$$m[i, j] = m[i, k] + m[k+1, j] + (p_{i-1} \cdot p_k \cdot p_j)$$

$\downarrow$ $i=1, 1 \leq k \leq 4, j=5$ fine calcolo

$K=1, 72 \leftarrow$

• SOL OTT, ma



4. (1 punto)

Si converta la seguente espressione da forma prefissa a forma postfissa:

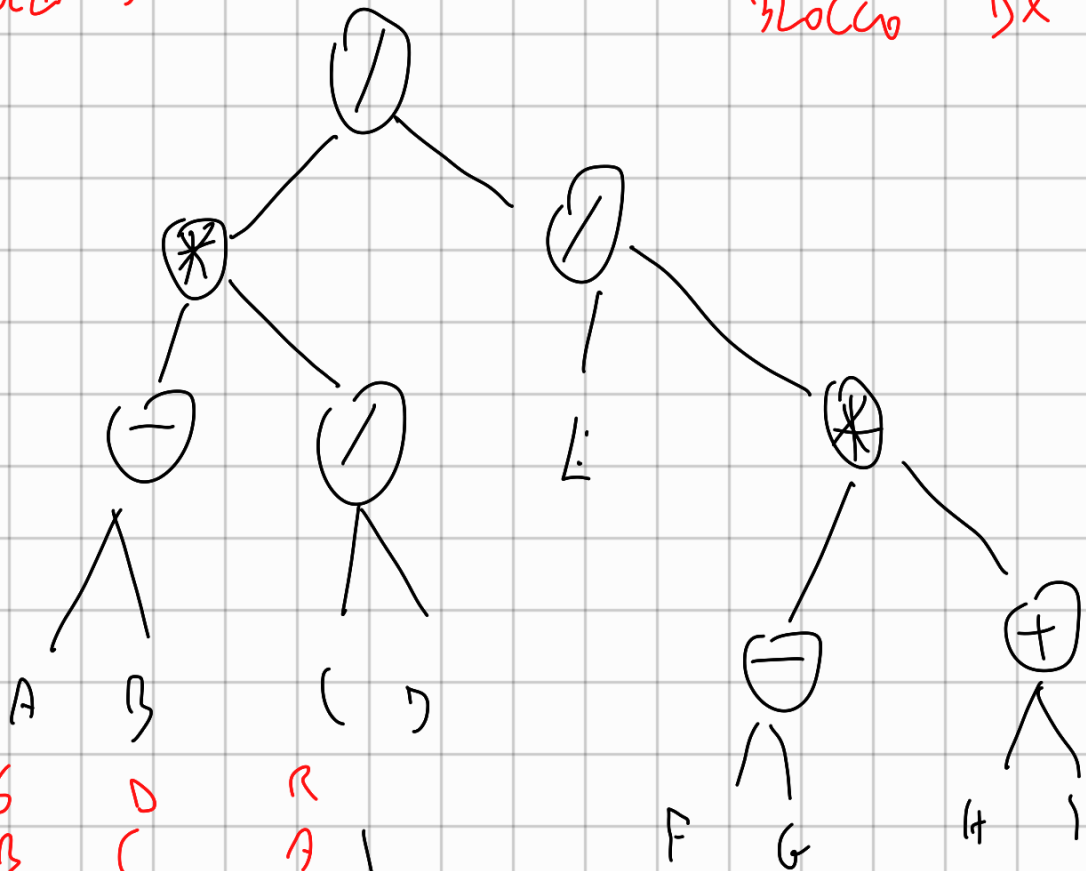
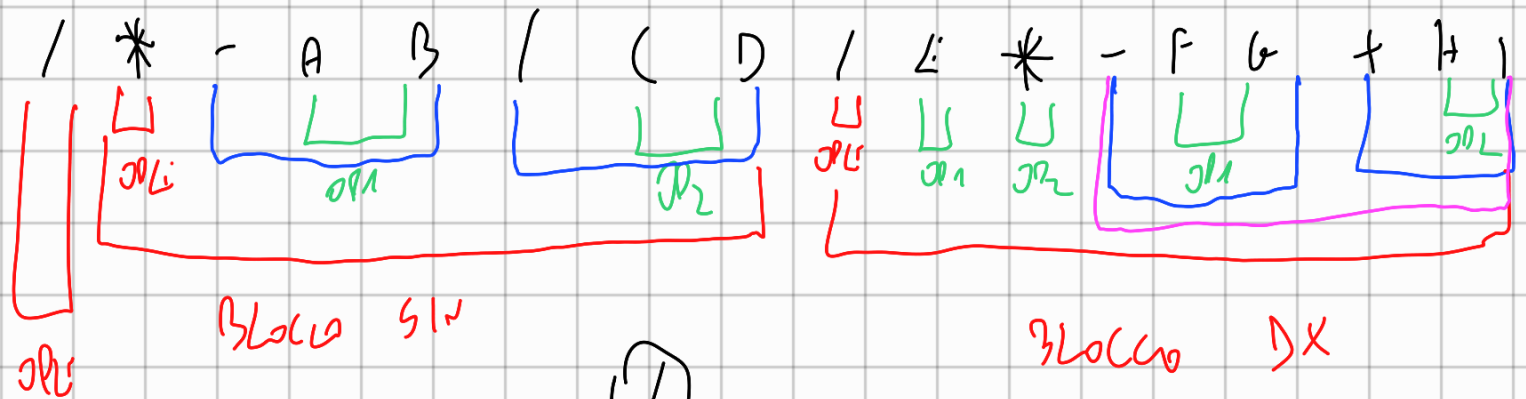
$/ * - A B / C D / E * - F G + H I$

Espressione in forma prefissa, infissa e postfissa

Tutti i nodi che non sono foglie sono operatori. Le foglie sono i valori utilizzati in combinazione con gli operatori.

1. La forma prefissa è definita attraverso una visita pre-order di un albero delle espressioni.
2. La forma infissa è definita attraverso una visita in-order di un albero delle espressioni.
3. La forma postfissa è definita attraverso una visita post-order di un albero delle espressioni.

Modi operari: $\left| \begin{array}{c} A \quad B \quad C \\ \text{OP}_1 \quad \text{OP}_2 \quad \text{OP}_3 \end{array} \right| \left| \begin{array}{c} B \quad C \quad A \\ \text{OP}_1 \quad \text{OP}_2 \quad \text{OP}_3 \end{array} \right|$

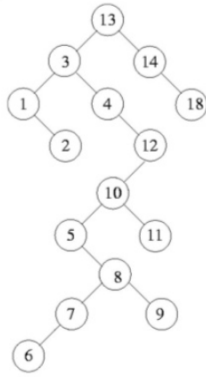


Post: $\left| \begin{array}{c} A \quad B \quad C \quad D \quad E \quad F \quad G \quad H \quad I \\ \text{OP}_1 \quad \text{OP}_2 \quad \text{OP}_3 \end{array} \right|$

$A \ B \ - \ C \ D \ / \ * \ E \ F \ G \ - \ H \ I \ + \ * \ / \ /$

5. (2.0 punti)

Si partizioni il seguente BST attorno alla sesta chiave più piccola. In ogni nodo compare la chiave intera, altre eventuali informazioni non sono riportate.



Partizionamento intorno alla k-esima chiave

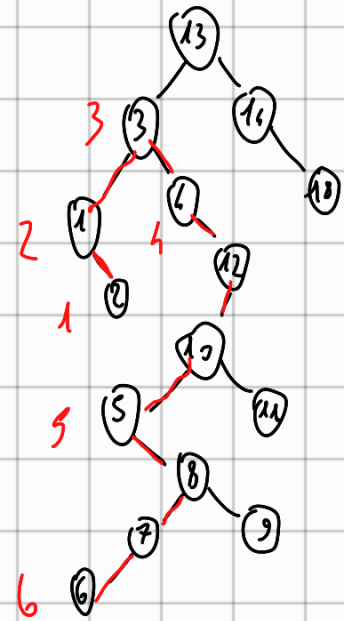
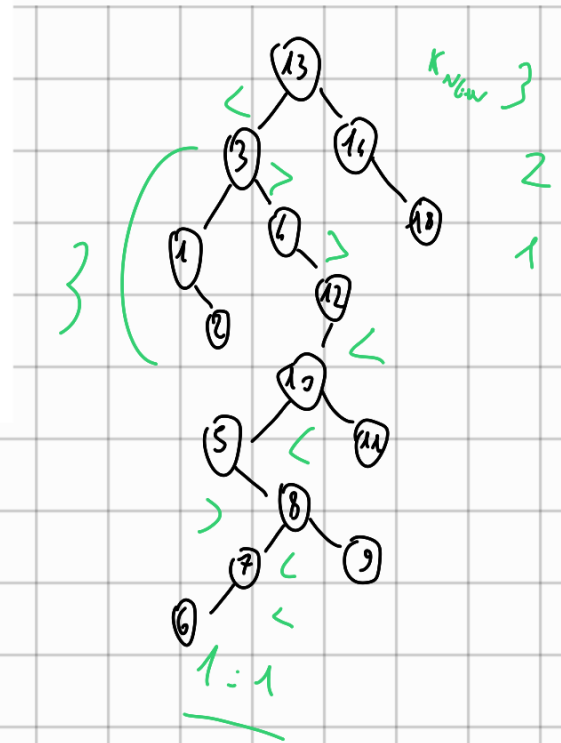
1. Si ricerca la chiave di rango k .
2. Si effettuano rotazioni coerenti fino a portare l'elemento alla radice.

$k: 6$

Selezione di una k-esima chiave

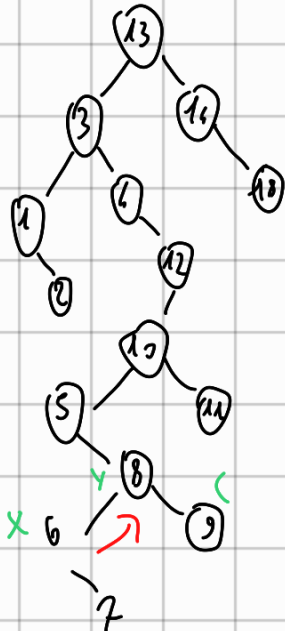
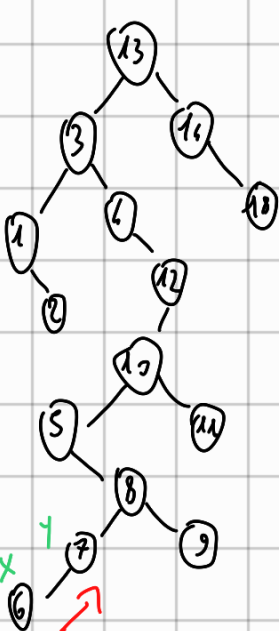
Per la selezione è necessario avere un BST esteso, con la cardinalità presente in ogni nodo e i nodi sentinella alle foglie.

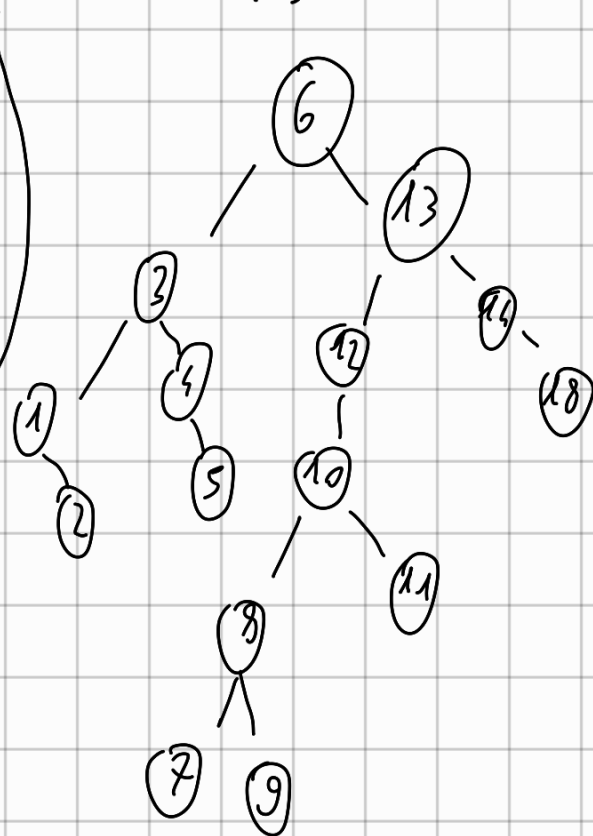
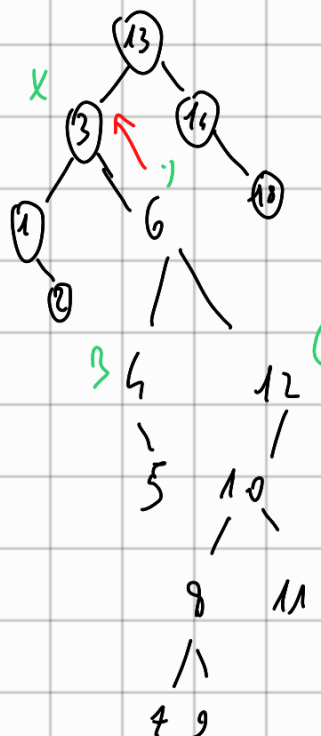
1. A partire dalla radice si effettua il confronto tra il rango e la cardinalità del figlio sinistro, dove c è la cardinalità del figlio sinistro.
 - a. Se il rango è maggiore della cardinalità, allora si procede nel figlio sinistro con rango k .
 - b. Se il rango è minore della cardinalità, allora si procede nel figlio destro con rango $k = k - c$.
2. Si ripete fino ad avere $k = c$.



$6 = 6^o$ Uniquo: il più piccolo

Partizionamento: l'aria 6 in radice

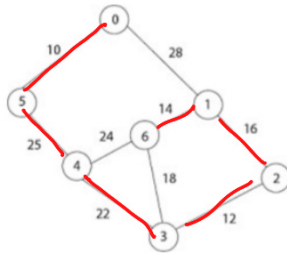




Final

6. (2.0 punti)

Dato il seguente grafo non orientato, connesso e pesato, se ne determini un minimum spanning tree applicando l'algoritmo di Prim a partire dal vertice **0**, disegnando l'albero e ritornando come risultato il valore del peso minimo. Si esplicitino i passi intermedi.



MST: $10 + 25 + 22 + 12 + 16 + 14$
 $(0,5) (5,4) (4,3) (3,2) (2,1) (1,6)$

599

Algoritmo di Prim

1. A partire da un determinato nodo, si effettuano i tagli sui nodi che raggiungono nodi adiacenti non ancora visitati.
2. Si sceglie, per ogni taglio effettuato, l'arco a peso minore.
3. Si ripete fino ad avere un MST.